Box Model Basics

So far in this class we discussed using cascading stylesheets (CSS) to stylize the visual look of the HTML within a webpage. This includes stylizing type and backgrounds.

In this lab we will begin to use CSS not just as a way to create visual style, but also as an important layout tool. This includes laying out placement of HTML elements and adding elements of spacing.

In this lab we will be take an existing webpage for a client named “Karla’s Cookies”. We will apply basic CSS box model properties to assist with spacing and placement.

Setup

Begin by opening *box-model.html* into your code editor. Take a moment to look it over and you’ll see that we have text and images located inside a master * * tag.



Open *style.css* and you can see that basic CSS styles have been setup. This includes CSS to stylize the text properties including color and font family. It also has set the background colors of the page and the * *.

Preview

Preview *box-model.html* into the browser and you should see something similar to the image on the right. We have text and images inside a main area with a light green background. The webpage itself has a deep forest green background.



Layout

What is layout? Well, layout is the act of positioning or “laying out” content such as text, images and media within a document.

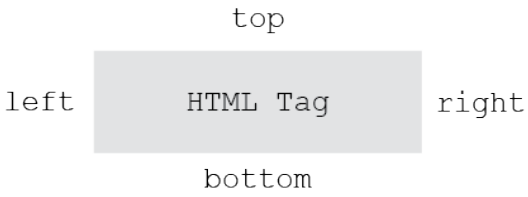
Many of you have taken classes where you have performed layout for print or digital distribution using programs such as Adobe Illustrator or Adobe InDesign. In those programs you had a drag-and-drop interface to assist with layout and position content. In web development we can also perform layout, but instead of using a drag-and-drop interface we’ll be using CSS.



© David Hurwich

Box Model Basics

To apply layout elements such as dimensions and spacing we must understand a CSS concept known as the box model. The box model concept is the idea that each HTML tag can be thought of as a box or rectangle. For instance the * * tag in our HTML can be thought of as a two dimension box. Each "box" has layout properties associated with it. This includes properties such as dimensions, borders and spacing. In the box model concept boxes also have sides. Each box has four sides: the top, the bottom, the left and the right.

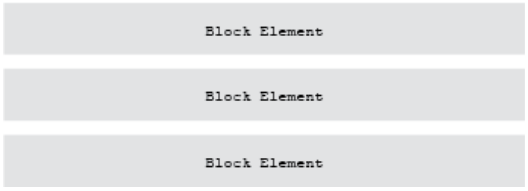


Dimensions

To create a good layout, we need the dimensions of items within the layout. This helps us organize and layout items to maximize space.

By default HTML items do not have dimensions such as width, but we can add those using the box model property '*width*'.

By default the width of an HTML tag depends on whether it is an inline element or a block element. Remember that is an inline HTML element displays in a line and a block HTML element displays as blocks vertically on top of one another. If you unsure whether an HTML tag is inline or block check Mozilla's reference list here:



Inline Element List: https://developer.mozilla.org/en-US/docs/Web/HTML/Inline_elements

Block Element List: https://developer.mozilla.org/en-US/docs/Web/HTML/Block-level_elements

If we look at the page we can see that the lightgreen container holding the text/ images for Karla's Cookies is the width of the webpage. If you resize your browser you'll see the container always resizes the container to fit the webpage. The reason for this is the container is a * *. s are block elements and by default the width of a block HTML element is 100% of it's direct parent. In the lab, the 's parent is the *<body>* tag or the actual webpage itself. Therefore, when you resize the webpage, you resize the * * accordingly. Let's override this default by applying a CSS width property.



In *style.css* set the following at the top of the stylesheet:

```
div {
 width: 600px;
}
```

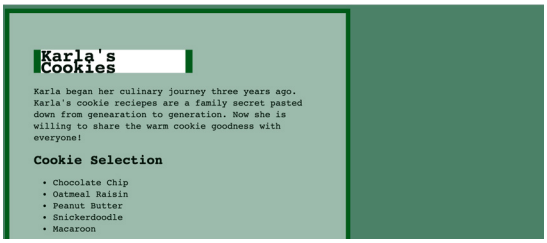


© David Hurwich

Save and preview and you see that the container is no longer the width of the page, but locked to 600px wide. If you resize your browser you'll see it no longer readjusts itself.

Heading

If you look at the webpage you can see that the heading or *<h1>* has a white background. Let's resize this using the our new CSS width property.



Set the h1 to the following:

```
h1 {
   width: 50%;
}
```

Save and you'll see that the heading backing is shorter. Like the * * the *<h1>* tag is also a block element. As a block element by default the *<h1>* takes up 100% of it's parent, which is the * *. By setting it's width to 50% we set the *<h1>*'s width not to the width of the webpage, but to the width of the * *.

To see this in action, let's change the *width* of the *div*. Change the width from *600px* to *800px*.

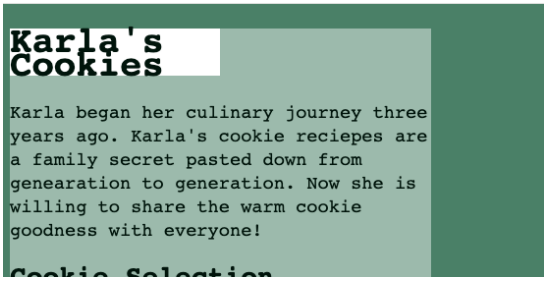



```
div {
   width: 800px;
}
```

Save and preview and you'll see not only did the * * container get wider, but so did the *<h1>*.

Max-Width

Instea of having the div container by a finite exact amount like 600px or 800px, let's make it a percentage of the webpage iteself. This will make the design a little more flexible.

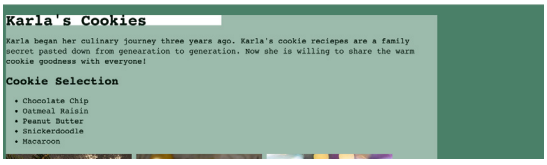


Set the div to a width of 80%

```
div {
    width: 80%;
}
```

Save and reload the webpage into the browser. Resize the browser and the main container should resize as well.

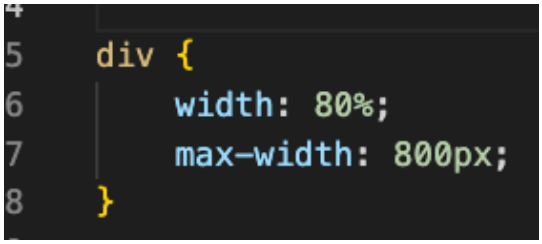
Although having a design resize based on percentages can be nice, there are drawbacks as well. If the webpage gets too wide that text becomes hard to read. Too prevent this we can set a *max-width*, this means that the width can not becomes greater



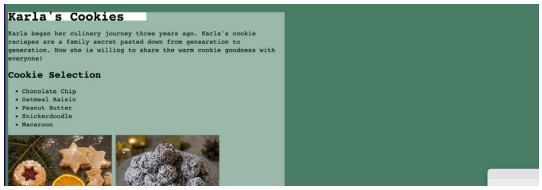
© David Hurwich

than a certain value. Add the *max-width* CSS property to the *div* and set it *800px*.

```
div {
    width: 80%;
    max-width: 800px;
}
```



This means the div will normally be *80%* of the webpage until it reaches *800px*, then it will no longer grow.



Save and preview the webpage in a browser. You will notice that when the webpage is thin the *div* is *80%*, but when the webpage gets wide it becomes locked to *800px*.

Fit-Content

The CSS property is a powerful tool we can use to set a specific width, either based on finite values like pixels or relative values like percentages. However, one issue with setting widths with the *width* property is that we always need to know the exact value. Sometimes we may not know the exact width we need, or we need a width that can update.

The solution to this problem is a value called *fit-content*. The *fit-content* value when set to width will make the width of the HTML container fit to the content.

Set the following in the *style.css* document

```
footer {
   width: fit-content;
}
```



Save and preview and you will see that the width of the footer is now exactly the length of the text content, no more, no less.



Let's test this by editing the content of the footer. Open *box-model.html* and change the text in the *<footer>* tag from *"Copyright Karla's Cookies"* to *"Copyright Karla's Cookies - A Globochem Company".* Save and you will notice that the width of the footer extends with the longer text.



© David Hurwich

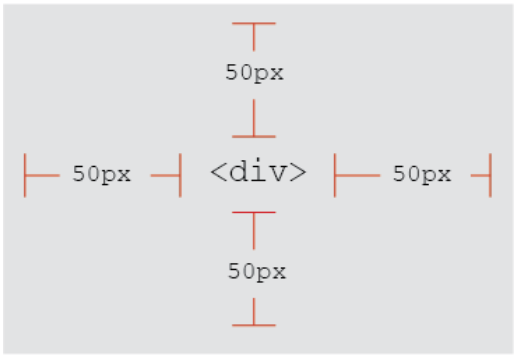
Padding



If we take a look at our webpage the content of the website, the text and images are right up against the edge of the container. Although this is wrong in terms of coding, it not great in terms of visual design. We want the text to have room to breathe. To assist with this we will use the box model property '*padding*'. Padding is the space between the content and the edge of the HTML tag itself.

Let's begin by giving some space between the text and images and the container. Apply the following:

```
div {
   width: 800px;
   padding: 50px;
}
```



Save and preview and you'll see that space has been added between the container and content. In fact, what has happened is that 50px of padding has been added to the four sides of the * * tag. 50px to the top, 50px to the bottom, 50px to the left and 50px to the left.

Borders

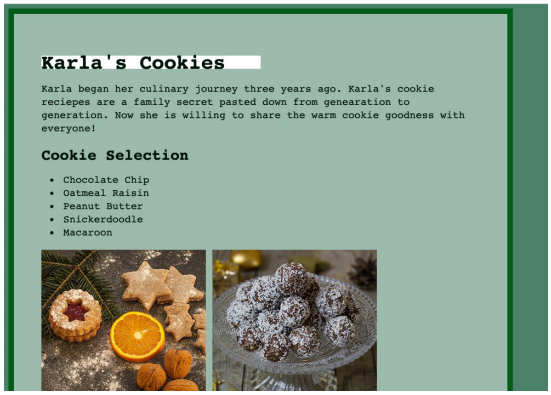
A border is a design that goes around the perimeter of our HTML container. If you have ever used vector programs like Adobe Illustrator or Adobe InDesign a border in CSS is very similar to a "*stroke*" in one of those programs.

Borders are useful as a design element. They help to contain or create visual separations in our designs and work.

Let's add a border to the Karla Cookies container. This will create an interesting design that will create a visual separation between the text and images and the rest of the webpage.

Add the following to our * *

```
div {
   width: 800px;
   padding: 50px;
   border: solid 10px #005c1a;
}
```



Save and preview and you'll see that a border now goes around your container.

The CSS border property takes three properties: the style of the

© David Hurwich

border, which in this case is solid, the thickness of the border which in this case is 10px thick and the color of the border, which in this case is green specified by a hexadecimal value. By default the border will apply to all four sides of div, the top, the bottom, the left and the right.

There are many styles of borders you can use. Refer to the class slides or use this online reference to view all of the possible options.

https://www.w3schools.com/css/css_border.asp

When listing the border properties the three properties can go in any order.

Box-Model Side

So far when applying padding and borders we've applied the property to all four sides of the HTML tag: the top, the bottom, the left and right. With CSS box model properties we can apply the setting to just one side if we want, instead of all four.



To see this in action, apply the following:

```
h1 {
   border-left: 15px solid #005c1a;
}
```

Save and preview. You'll see that it applies the border to the *<h1>* tag, but only on the left side of the container, not to all four.



Let's add CSS to apply the same border settings to the right side of the *<h1>* tag.

Save and preview and it should look roughly like the image on the right.

Images

In addition to affecting HTML containers we can apply box-model properties to elements such as images, audio and video. Let's use CSS box model properties to resize the images in our webpage. At the moment the images are very large, we'll use CSS to resize them smaller and add a border.

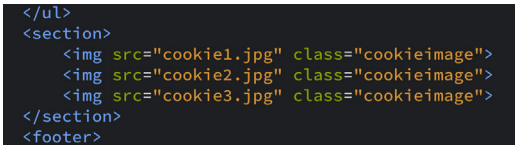


Although we could use the *img* selector in the CSS document to select our images, we may not necessarily want to affect ALL of our images, so let's create a class instead. Remember that a class is a selector that us the developer's create, then we can apply them inside our HTML code.

© David Hurwich

Let's begin by creating a class called *.cookieimage* inside our *style.css* document. Let's use the CSS *width* property to scale our images smaller.

```
.cookieimage {
  width: 200px;
}
```



After creating the class go into the *box-model.html* document and apply the class to the three images at the bottom of the * *. The code should look roughly like this:

```
<img src="filename" class="cookieimage">
```



Apply this to each image. Save and preview and you should see the images on the webpage resize to be smaller.

Now that we've resized the images let's add a border / frame around the image. Add a CSS *border* property to *.cookiemage* that sets the border style to: double, 10px width with a color of *#005c1a*.



Save and the webpage should look something like the image on the right.

Margins

Earlier in the lab we used padding to add space between the content and the edge of the HTML tag. Now we are going to apply spacing to position HTML tags from other HTML tags. To do this we'll be using the CSS property margins.

Take a moment to look at webpage. Notice that the footer copyright text is very close to the images. If you look at the HTML you'll see that the *<footer>* holding the text is below the *<section>* containing the images. Let's use CSS margins to add a little space.



Use the *footer* selector and add *50px* of margin around it. The code would look like this:

```
footer {
    margin: 25px;
}
```

Save and you'll notice that it pushes the *<footer>* down from the section, the margin is acting as a buffer. To see this visually in Chrome right-click and choose "inspect', select the *<footer>* inside the Elements section and you'll see a peach color around the *<footer>* tag, that is the margin.



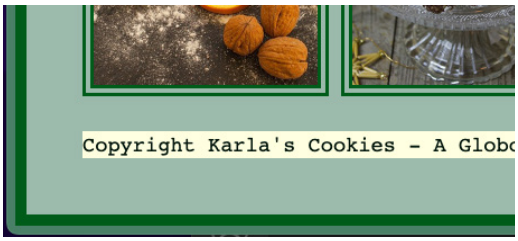
Notice that the margin is pushing the *<footer>* tag down from the

© David Hurwich

<section> tag, but it is also pushing the *<footer>* on the left and right side as well. This is causing the footer text to move closer to the center of the page. The reason for this is because not only does margin push the HTML tag from tags above and below, but it also pushes off the parent tag, which in this case is the * * tag.

In this case, we don't really want to push off the * *. The solution is rather than using the default CSS margin that applies to all four sides, let's just apply the margin to the margin-top for the *<footer>* tag. Rewrite the code to look this like this:

```
footer {
    margin-top: 25px;
}
```



Save and preview. It should roughly like the image on the right.

Zeroing Out

Hopefully now you can see that padding, borders and margins can help us create a layout in our webpage. It is important to understand that the HTML webpage has some padding and margins applied by default when previewed in a browser.

Have you ever wondered why there is space after a heading HTML tag like a *<h1>* or *<h2>* and the content that comes afterwards? The answer is because the browser has default CSS applied to these tags. The default CSS are using margins and paddings to space out elements.

If you look at *Cookie Selection* text in our webpage preview you'll see there is space between the text and the cookie list. If you right-clicked on the text in the Chrome Browser and choose *Inspect Element* you can see that *<h2>* has margin applied to it by default above and below it.

Let's remove the margin applied on the bottom of the *<h2>*. Without the bottom margin the text and the cookie list will be closer to each other. To do this we'll use a method called '*zero-ing out*'. Zero-ing out means to remove default CSS applied to an HTML tag, in this case the *<h2>* tag.

Add the following:

```
h2 {
   margin-bottom: 0px;
}
```

Save and preview and you see that the *<h2>* gets closer to the unordered list. That is because we overrode the margin on the

© David Hurwich

bottom of the *<h2>* by setting it to margin-bottom of 0px.

Now that we've fixed the *<h2>,* let's remove the space on the left side of our unordered list. By default unordered lists have padding on the left side, this is what causes the default indent. To remove the indent we'll override the padding on the left side of the *<ul>* tag.

Add the following CSS to *style.css:*

```
ul {
   padding-left: 0px;
}
```



Save and you'll see that the bullet points and the text has moved over to the left. That is because we overrode the default padding on the left side of the list.

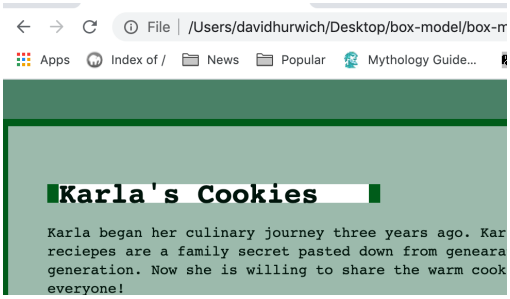
Finally, let's look at the webpage itself, the *<body>*. If you look at the top left hand corner you'll see that container is pushed off of the corner. That is because the *<body>* itself has a default margin applied.



Go into the CSS and zero out the *<body>,* that is say set the margin for the *<body>* to be zero. Save and preview and your webpage should look like the image on the right.

Margin Positioning

Finally, let's use margins to help position the Karla's Cookies container. Right now the container is at the upper left hand corner of the page. Let's move it down a bit by applying a bit of *margin-top* to the * * container.



Add 50px to the margin top of the * * tag.

Now let's use *margin-left* to position the container to the center of the page. Apply a *margin-left* to the div of 250px. See the code below:

```
div {
   margin-top: 50px;
   margin-left: 250px;
}
```



Save and you'll see that the container is closer to the center of the webpage. The problem with this method is that your container will *ALWAYS* be 250px from the left side of the page. If you resize your browser you'll see that the container does not move and will farther on the left or farther on the right side of the page,

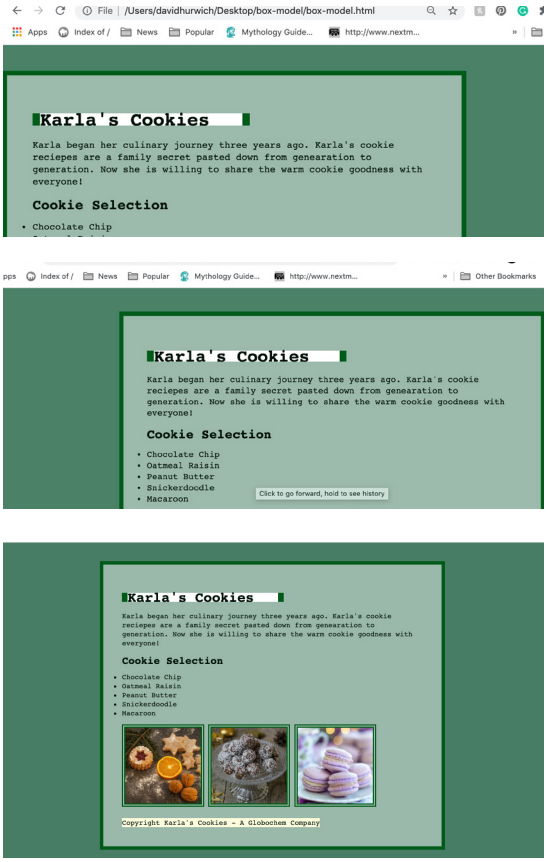
© David Hurwich

depending on how big your browser is.

The solution to this problem is to use a value of *auto*. What *auto* does it is tells the browser to apply all of the available space in the webpage to the margin. If we set the *margin-left* to *auto* it will apply all the space to the left side of our container. Save and preview, you'll see that the container is placed on the right side of the browser, and the all of the available space on the left. If you change *margin-left* to *margin-right* and set it to auto as well you'll get the same result, but with the container on the left side instead of the right side.

But, how does that fix our issue, we want the container in the horizontal center of the webpage? The answer is to apply BOTH. If you have margin-right AND margin-left set to auto both sides. This will split the difference and make sure that the left side and the right side get equal amounts of the available space. The result of this will be placing the container in the horizontal center of the webpage.

Save and preview and your webpage should look roughly like the image on the right.



Submission

Zip the folder containing all the necessary files including HTML, CSS and image files.

Submit the folder to Canvas

Extra Credit

Add additional box model properties to Karla Cookies webpage.

- Add rounded corners to the * * tag using *border-radius*.
- Add a drop shadow to the * * using the CSS *box-shadow* property.
- Use the CSS *border-radius* property to make the images into circles.

© David Hurwich